

MCP and Custom Tools Enabled Personal Assistant

Department of Computer Science & Engineering and Information Technology
Jaypee University of Information Technology, Waknaghat, Solan – 173234, India

SUBMITTED BY

Nikhilesh Sharma (221030003)

Shashvat (221030117)

Yuvraj Saini (221030233)

SUPERVISOR

Prof. Dr. Pradeep Kumar Gupta

Professor, Dept. of CSE & IT

JUIT, Waknaghat

PROBLEM STATEMENT

Virtual assistants like Siri, Alexa, and Google Assistant answer questions and run simple voice commands, but they cannot work directly with the productivity tools people use every day — no editing of Google Docs, no live weather context, no developer lookups on Stack Overflow. LLM chatbots like ChatGPT and Gemini can reason well but cannot access user data without plugins. This project builds an assistant that combines an LLM with real APIs so it can carry out tasks, not just answer questions.

OBJECTIVES

1. Build an AI assistant that can complete real tasks, not just answer questions.
2. Connect a large language model to external APIs so it can perform work people normally do by hand.
3. Automate document handling, information retrieval, and routine digital tasks.
4. Use three task-specific agents (Student, Daily Life, Developer), each with its own tools.
5. Contribute to research on tool-enabled LLMs in a form that is also useful for schools and small teams.

IMPLEMENTATION DETAILS

The system is written in TypeScript on the **Mastra framework**, with **Gemini Pro 2.5** as the language model. The three agents share the same model but use different tool sets. LibSQL stores conversation context and OAuth 2.0 secures every Google API call. Each tool has a JSON schema the orchestrator checks before invocation.

- ▶ Mastra Framework
- ▶ Gemini Pro 2.5
- ▶ Google Docs & Drive
- ▶ OAuth 2.0
- ▶ Stack Overflow API
- ▶ Open-Meteo API
- ▶ Mermaid.js + Nari TTS
- ▶ LibSQL Memory

METHODOLOGY

The user types a query in plain English. Gemini Pro 2.5 reads it, and the Mastra orchestrator decides whether the model can reply directly or needs to call a registered tool. Tool calls are validated against a JSON schema, executed, and the result is fed back to the model for the final reply. OAuth 2.0 secures all Google services.

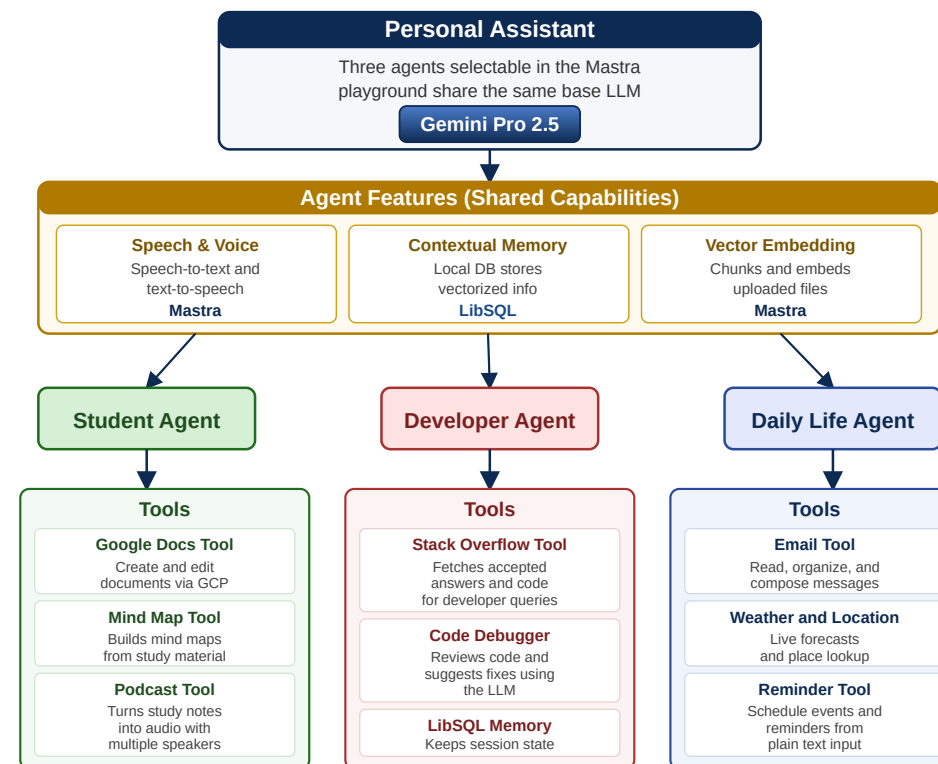


Fig 1. Multi-agent architecture with Gemini Pro 2.5 + Mastra

Design choices:

- One shared language model. Each of the three agents uses its own tools but the same Gemini model.
- JSON schemas for every tool. The orchestrator rejects calls that do not match, so the model cannot make up arguments.
- LibSQL memory. Document IDs and past replies are kept across turns of the same conversation.
- Limited API access. OAuth tokens are kept out of the code and each tool is given only the permissions it needs.

RESULTS & PERFORMANCE

Tested on 50 sample queries across the three agents. Reported benchmarks for tool-enabled LLM systems sit at 26–50 percent tool-success; this system reached 95.2 percent.

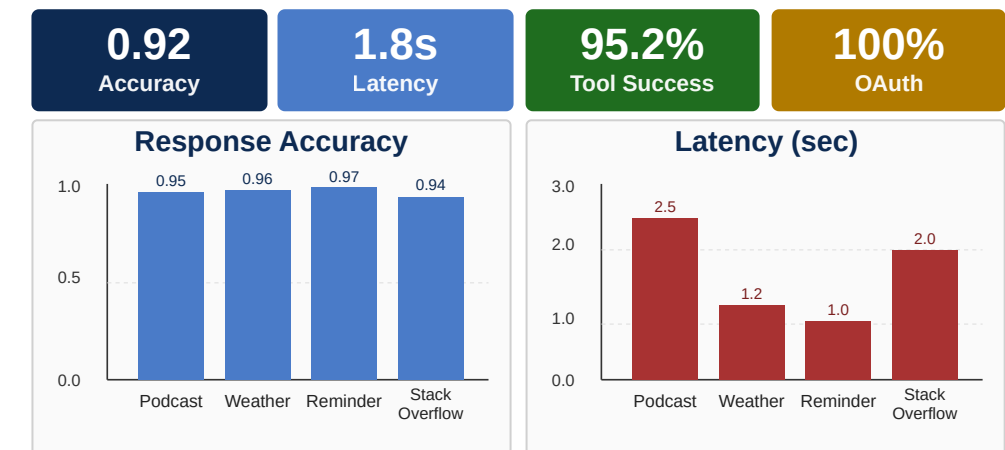


Fig 2. Accuracy and latency across tested tools

CONCLUSION & FUTURE SCOPE

Conclusion: The three-agent design reached over 90% tool success across every tool tested. All five objectives were met.

Future Scope:

- Long-term memory via vector database.
- Web/mobile interface with voice input.
- Caching to lower latency and API cost.

PUBLICATIONS

M. Mankotia, N. Sharma, Shashvat, Y. Saini, P. K. Gupta, **"MCP and Custom Tools Enabled Personal Assistant,"** JUIT Waknaghat, 2026. **Status:** Under Review.

REFERENCES

- [1] S. Hosseini et al., "Role of Agentic AI in Shaping a Smart Future," *Array*, vol. 26, 2025.
- [2] A. Singh et al., "SafeMate: An MCP-Based Multimodal Agent," arXiv preprint, 2025.
- [3] M. Hasan et al., "MCP at First Glance: Security & Maintainability of MCP Servers," arXiv preprint, 2025.